



NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

SC6019 – Blockchain Privacy & Scalability

Lecture 09

Neil HAN



Blockchain Scalability

Layer 2 Solutions: Off-Chain Scalability

Scaling Computation – Layer 2



Fundamental Problem in Blockchain Scalability

"every node process all TXs, store all data, plus total ordering of TXs is required"

What if we push TXs and state updates off-chain and only broadcast computation results on-chain?

Layer 1 scaling:

- Sharding
- Blockchain Interoperability
- Faster client implementation
 - compact tx/state encoding
 - parallelization

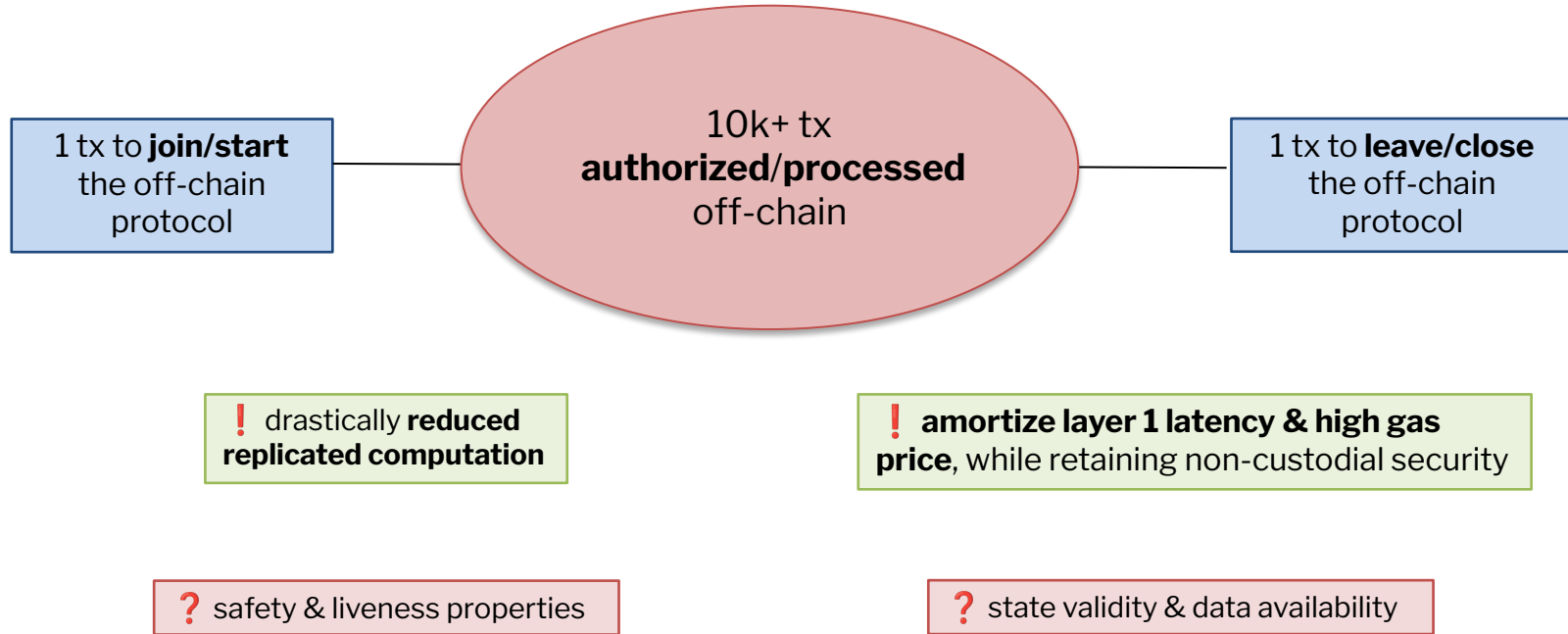
Optimizing "supreme court"

Layer 2 scaling:

- **Channel-based** approach
- **Commit-chain** approach

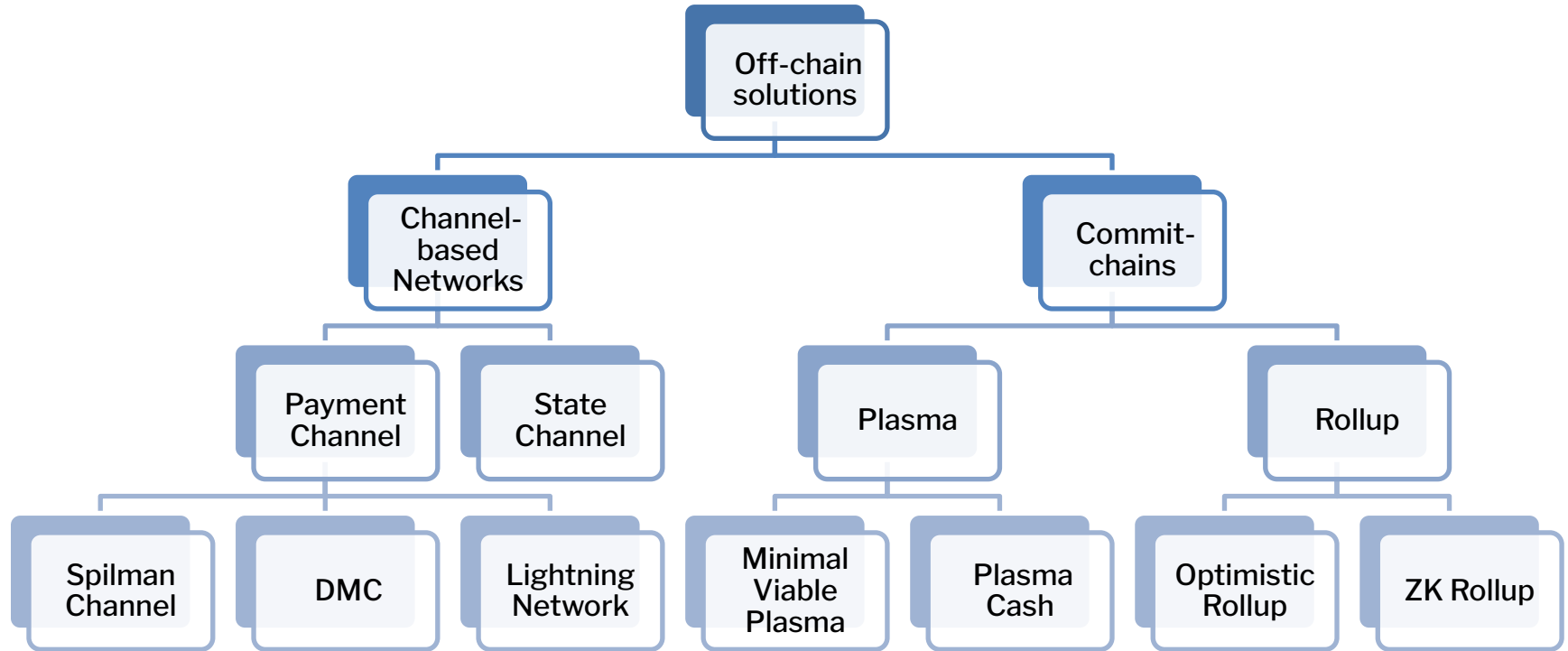
Offload work to "local court" and adjudicate only rarely during dispute

Why Layer 2 is Exciting ?



credit: Patrick McCorry's presentation on Offchain lecture: <https://docs.google.com/presentation/d/1VA5FM0Gtz-WXV-R6woqiPnCSapaMlsu5t-h8s5R-cyg/edit?usp=sharing>

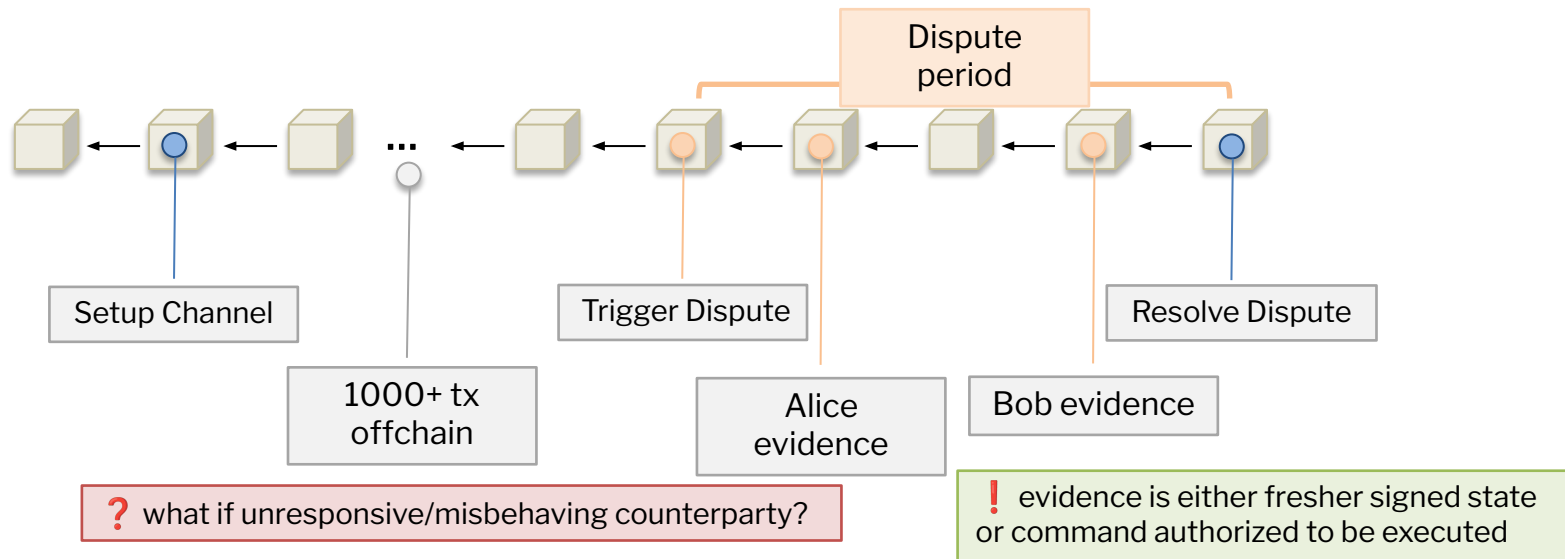
Promising Layer 2 Solutions



Blockchain Scalability

Scaling Computation – Channels

Generic Channel Workflow



Setup: a fixed group **initialize** states (e.g. deposit for initial balance)

Offchain progression: all parties cooperatively agree/sign on the new state (repeatedly)

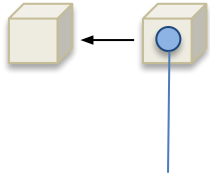
Onchain Dispute (challenge-response): honest party can always finalize/close channel with latest signed states, or force progression onchain with an unresponsive counterparty

Dispute Resolution: fast-forward to the latest state presented as dispute evidence, (can optionally close channel)

Payment Channel (mostly for Bitcoin)

- Replace by incentive -- Spilman Channel
 - One-way/unidirectional payment
- Replace by time lock – Duplex Micropayment Channel (DMC)
 - Bi-directional payment with expiry time
- Replace by revocation – Lightning Network
 - Bi-directional & no expiry
 - Using Hash Time Lock Contract (HTLC) to connect channels into a network

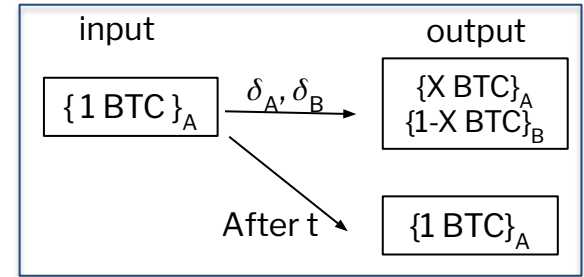
Spilman Payment Channel (unilateral micropayment)



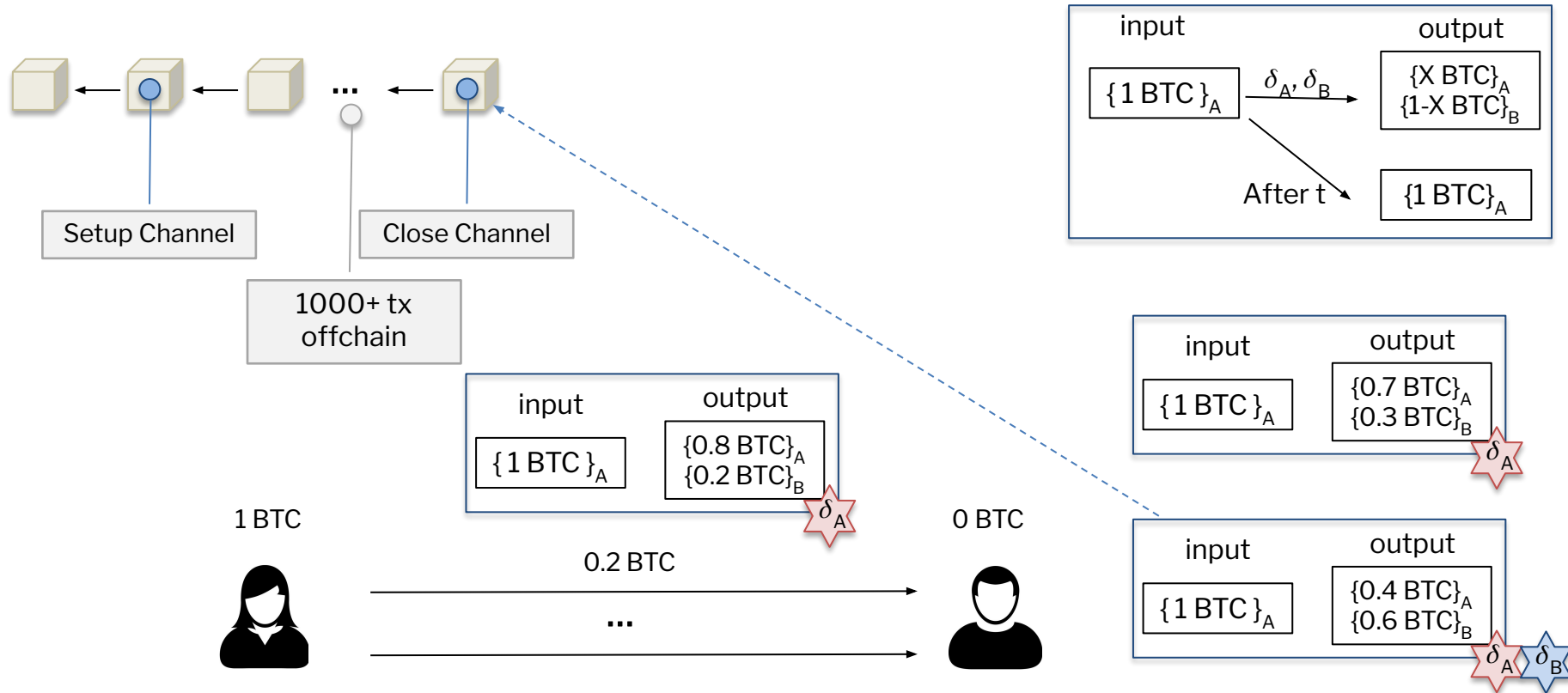
Setup Channel

Alice **deposit 1 BTC** which can be redeemed if **one of the two conditions** is met

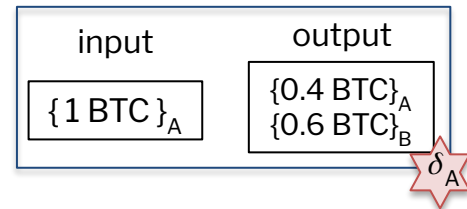
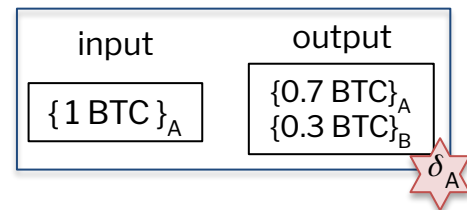
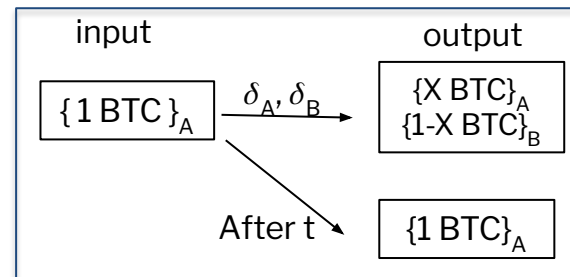
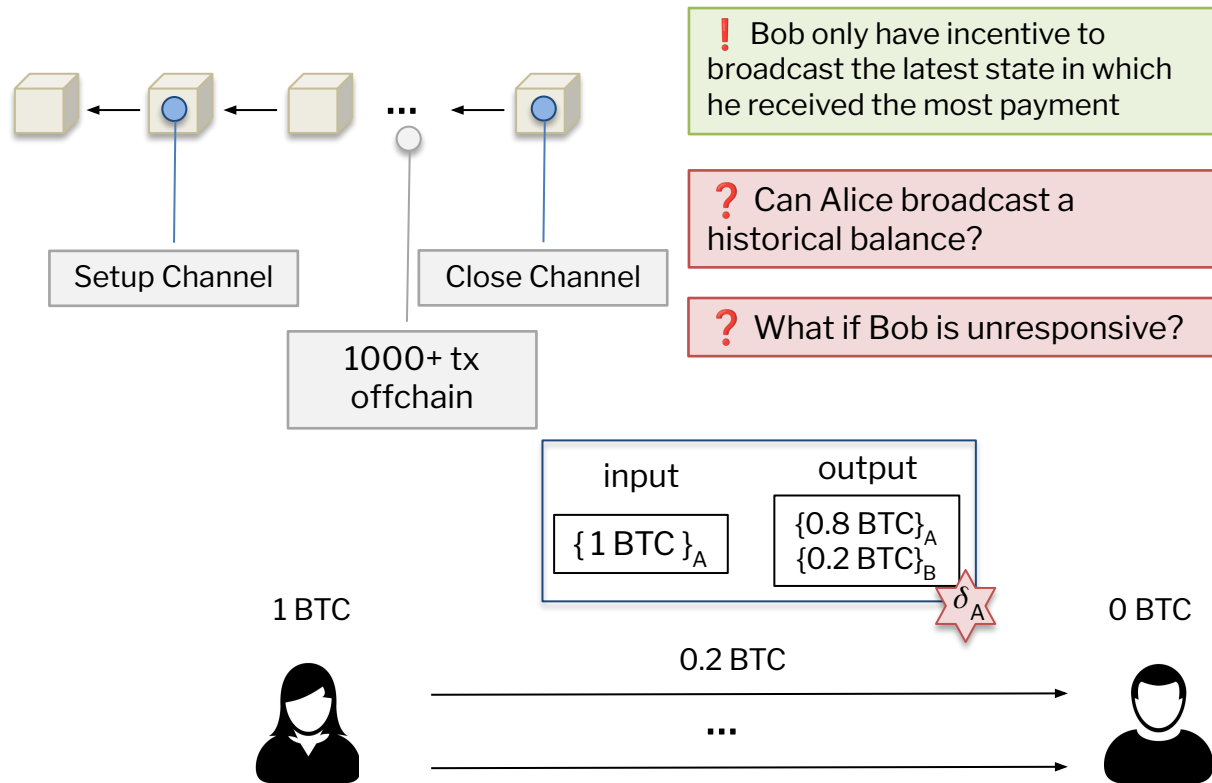
- A spend tx authorized by both Alice & Bob
 - After t timeout, Alice is refunded



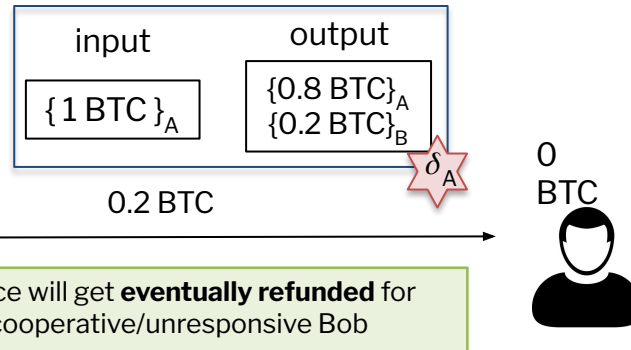
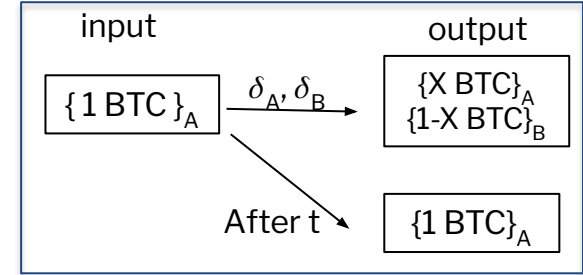
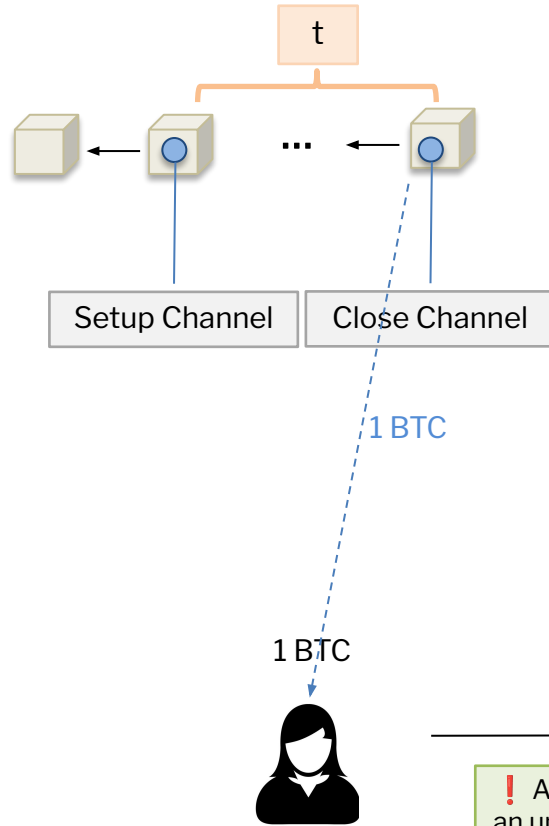
Spilman Payment Channel (unilateral micropayment)



Spilman Payment Channel (unilateral micropayment)



Spilman Payment Channel (unilateral micropayment)



Duplex Micropayment Channel (DMC)¹

What if we want bi-directional payment?

Core idea of DMC: 1 pair of unidirectional channel + validation tree

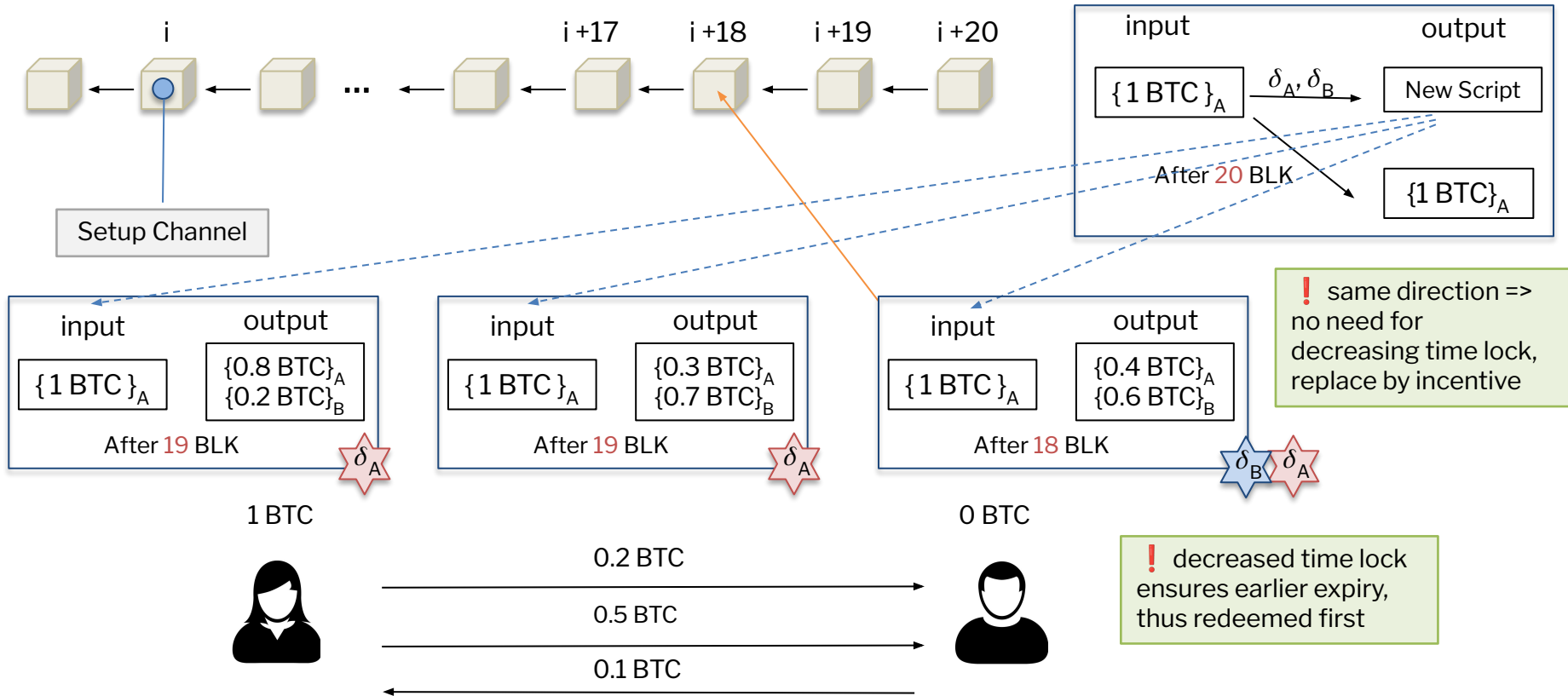
Core idea of "validation tree": Replace by time locks

Core idea of "Replace by time locks": decreasing expiry time for newer payment to ensure that the latest state will be redeemed first

Note: details of DMC protocol is consuming, and only of historical significance, thus will be skipped. In practice, Lightning Network >> DMC

[1]: paper at https://dl.acm.org/doi/10.1007/978-3-319-21741-3_1

Replace by Time Locks



Lightning Network [Poon, Dryja]

- **Instant finalization** – no more waiting for block confirmations
- **Low cost** – only 2 on-chain tx when cooperative, channel can keep open forever
- **Scalability** – one on-chain update = thousands of off-chain intermediate updates aggregated
- (Somewhat) **Private** – intermediate transactions are unknown to the world
- State **replacement by revocation** – avoiding restrictive expiry time in DMC while maintaining bi-directional payment
- A payment network via one types of *conditional transfer* – **Hash Time Lock Contract** (HTLC)

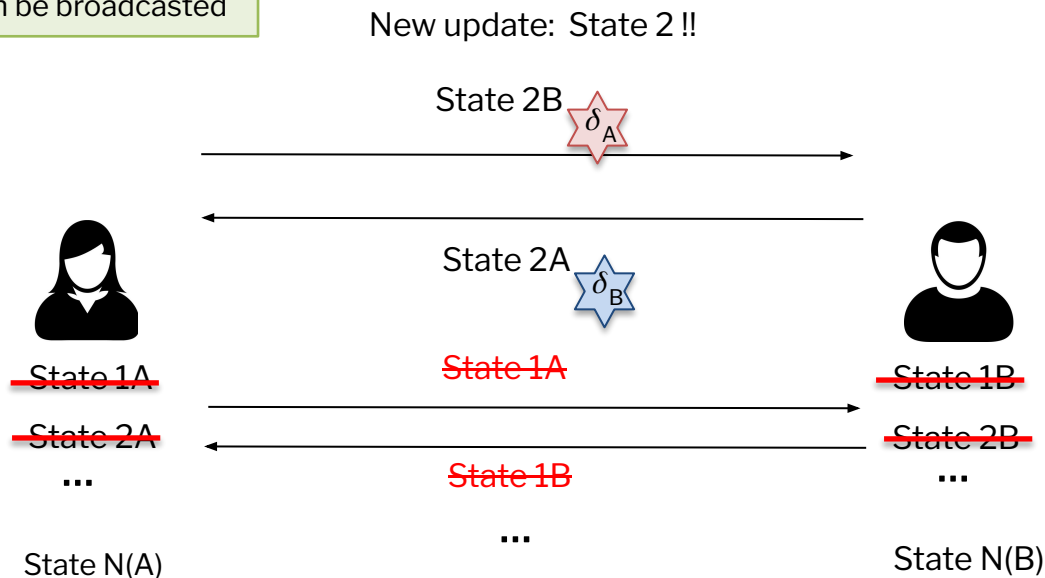
More at: <https://lightning.network/>

Lightning Payment: Core Idea

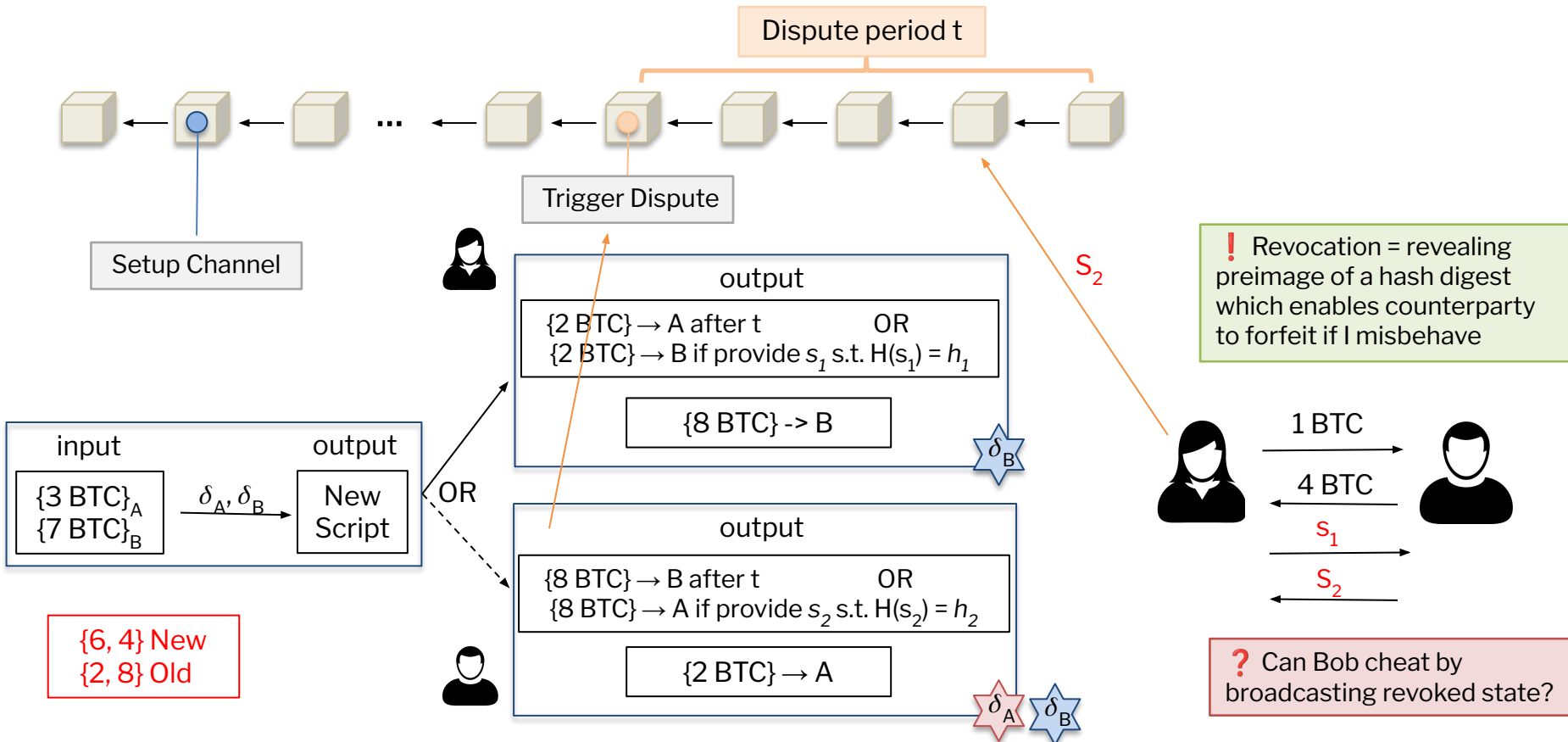
! For every state (redistribution of deposit), each party will sign/authorize a tx that only the counterparty can broadcast. When closing the channel, any one of the tx can be broadcasted

! authorizing a payment requires 2 steps
1. both authorize the new state
2. both revoke the old state

! If Alice try to broadcast a revoked historical state, Bob can present the revocation evidence to forfeit all of Alice's deposit as a punishment of misbehavior



Lightning Payment: Under the Hood



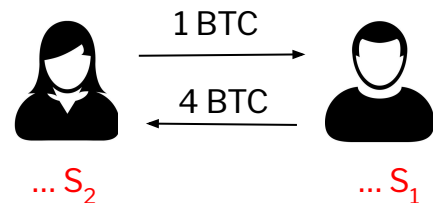
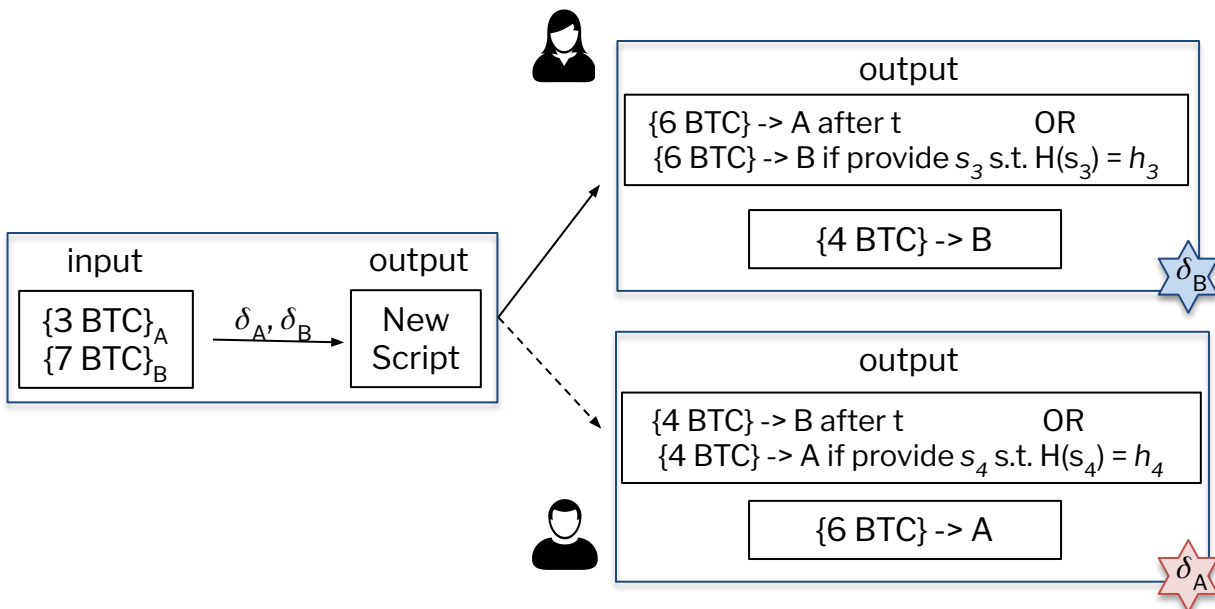
Lightning Payment: Under the Hood

State validity ? -- unanimous consent/multi-signatures + revocation to ensure "latest"

Data availability ? -- completely users' responsibility to store all historical revocation secrets & latest state

Safety ? -- counterparty can't steal in all cases

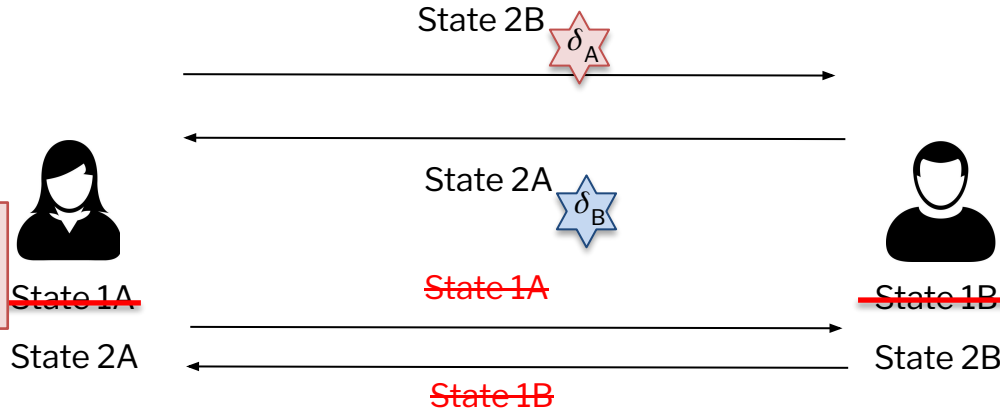
Liveness ? -- force a response from counterparty (or close channel) by triggering a dispute with latest state



Pause & Ponder

! Then Alice won't send revocation tx over (namely preimage of h_1), nor should Alice continue to sign further update
If Bob tried to close channel with State 2B, Alice is fine with it since she had agreed to it;
If Bob tried to close channel with State 1B, Alice is also fine, since she hasn't received State 2A, or accept State 2 yet

New update: State 2 !!

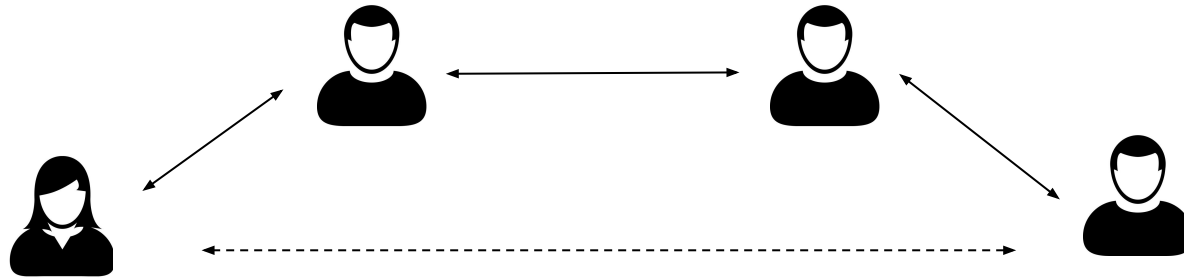


? What if malicious Alice refuse to revoke State 1A

? What if malicious Bob refuse to send over State 2A ?

! Then Bob should not accept State 2 yet, not until he received revocation of State 1A

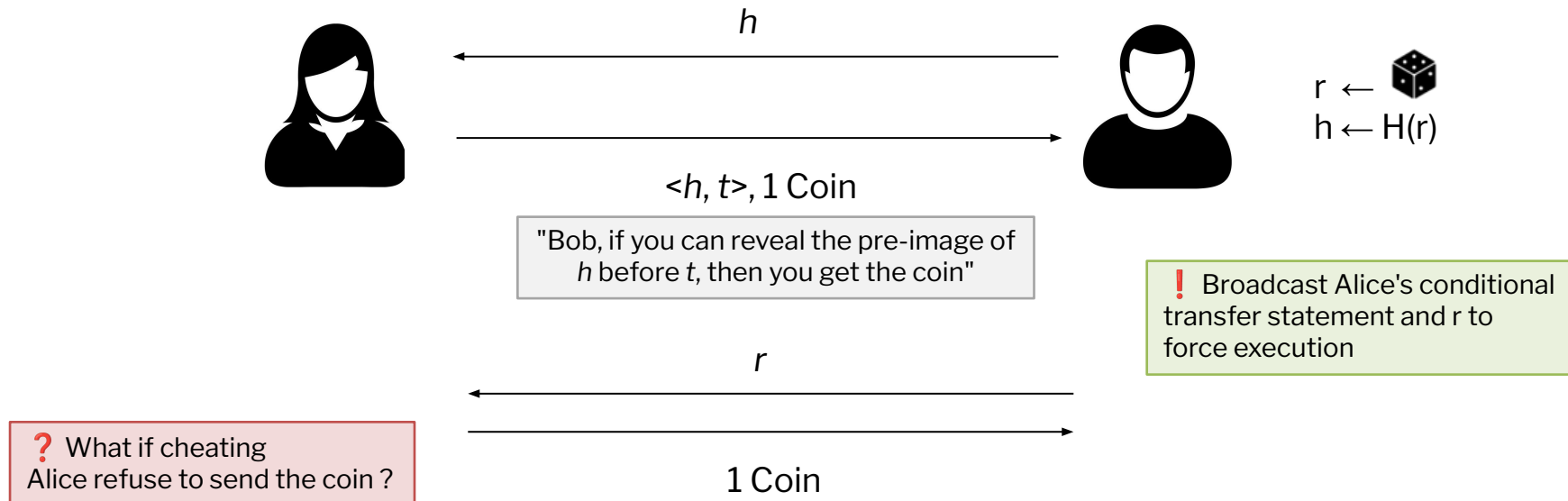
Pause & Ponder



? Can we set route payment through existing channels without creating new channel onchain?

Create a dedicated channel with everyone is expensive and capital intensive

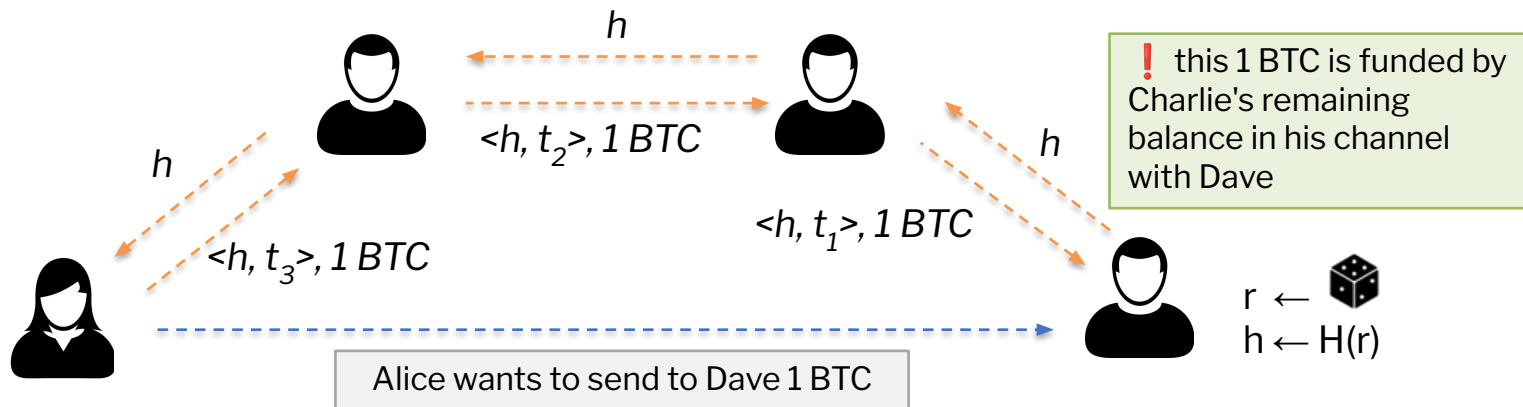
Conditional Transfer



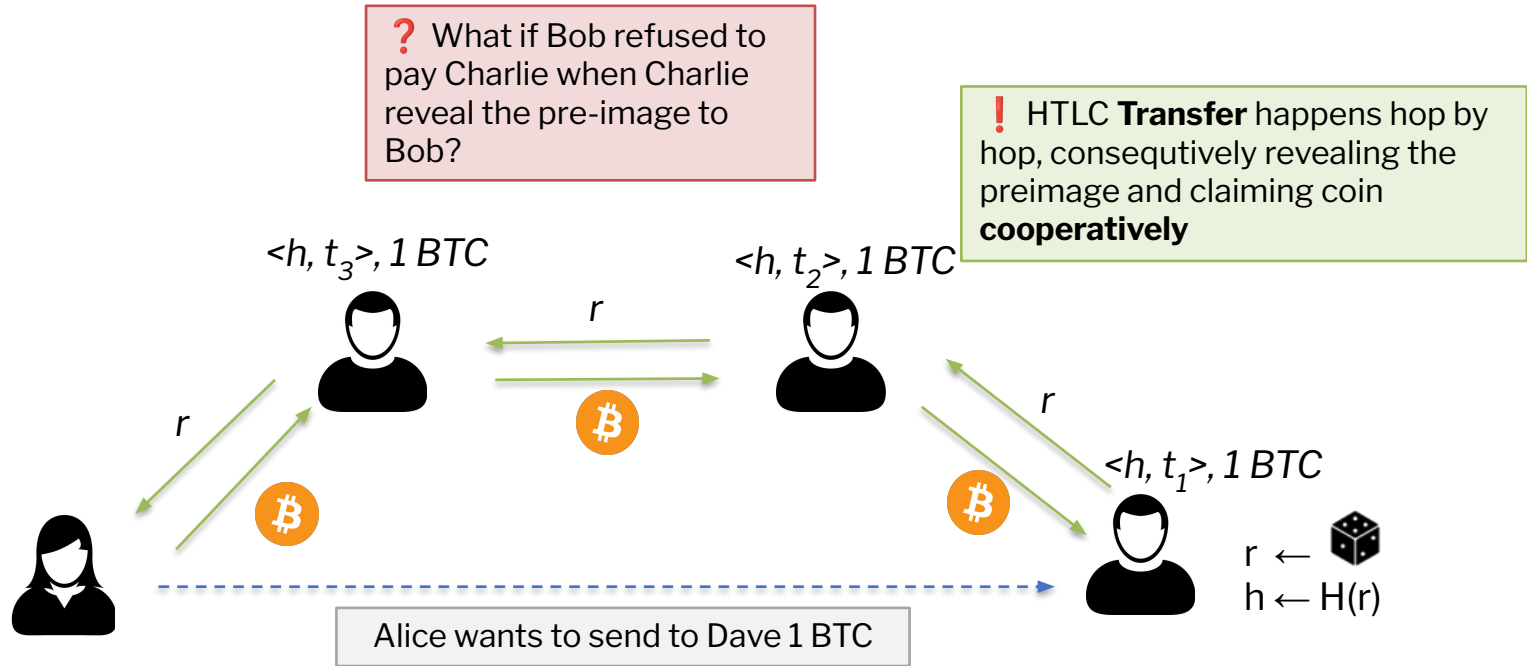
Hash Time Lock Contract (HTLC)

! **Setup:** from receiver (Dave) to sender (Alice), each hop commits to a conditional transfer on revelation of the preimage of the hash digest

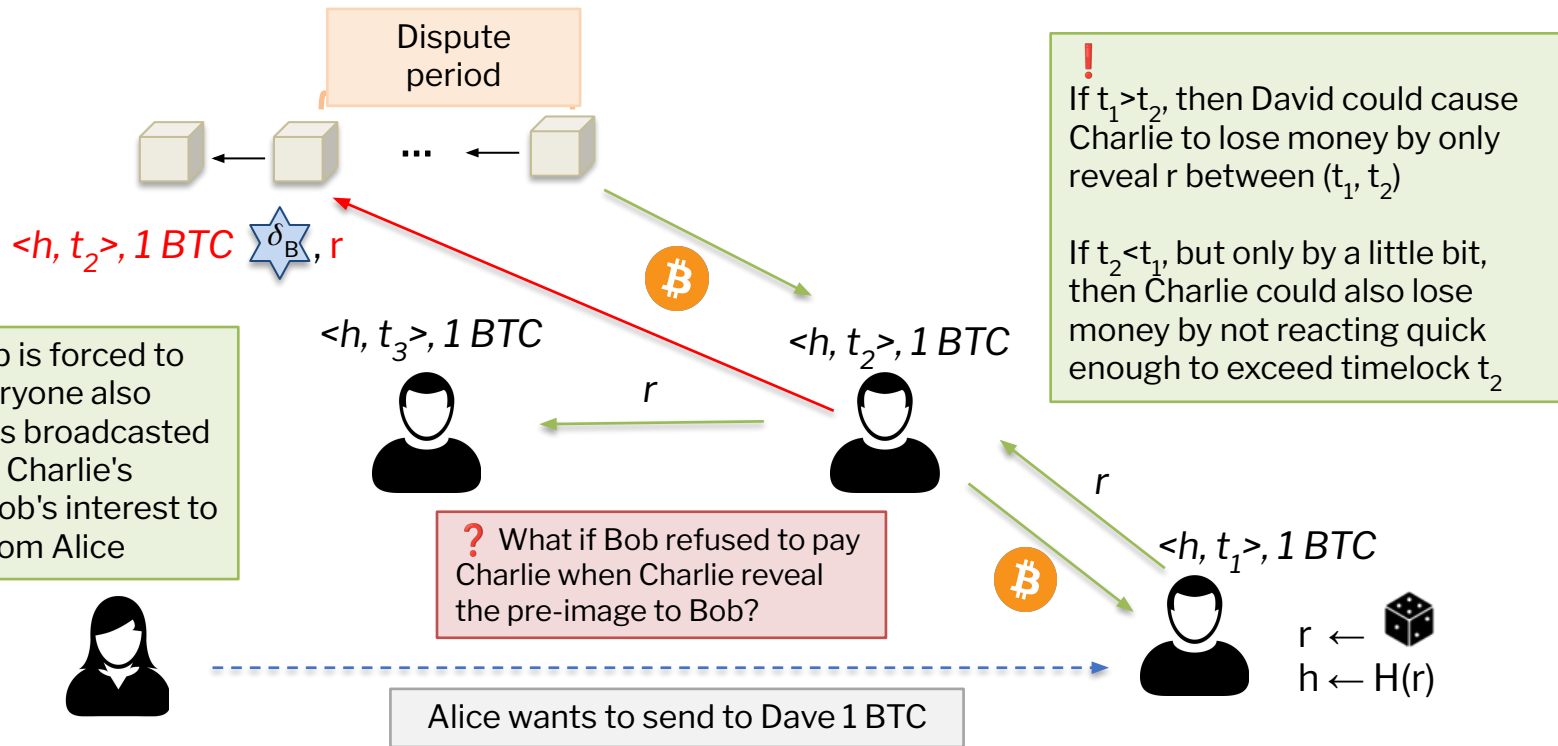
! **decremental timelock from sender to receiver** to give buffer for each hop to claim coin
 $t_1 < t_2 < t_3$



Hash Time Lock Contract (HTLC)



Hash Time Lock Contract (HTLC)



Lightning Network Analysis

Good:

- No need to set up new channel for each pair-wise transfer
- Routing marketplace, anyone can provide collateral and join
- Optimistically lightning fast

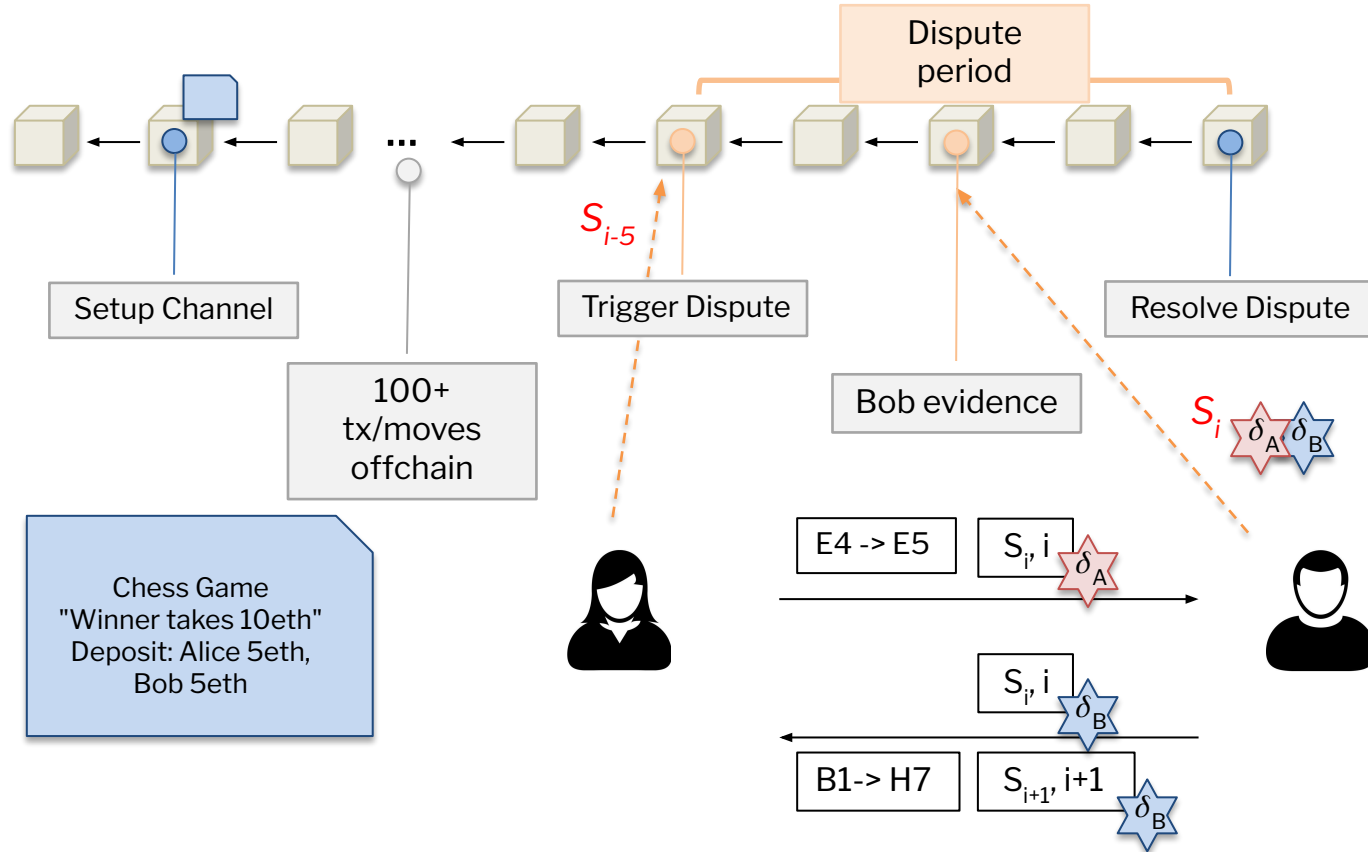
Bad:

- Free option to create and hold on to conditional transfer and waste liquidity
- Heavy collateral for multi-hop transfer (4 hops means 4X the collateral)
- Long worst-case resolution time for multi-hop transfer

State Channel

- Utilize the expressiveness of contract language in Ethereum to enable **arbitrary states** in a channel (including balance in a payment channel)
- **Replace by version** – each new state attached to an incremental counter with everyone's signatures
- **O(1) storage** – no more historical states bookkeeping

Application Specific State Channel

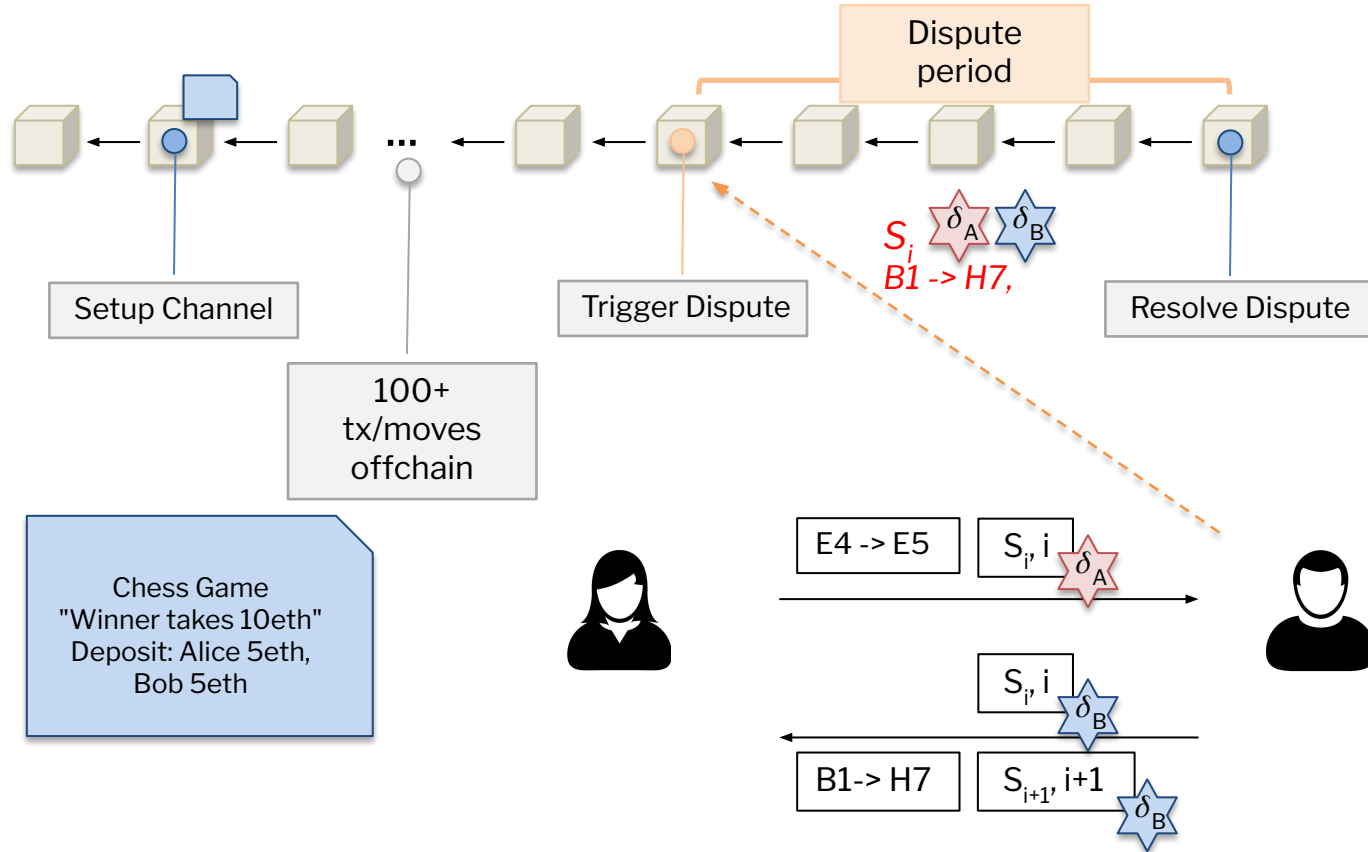


! $i > i-5$ && S_i signed by both parties $\Rightarrow$ Alice is cheating, forfeit all its deposit

? Alice regret S_i and try to close the game with old state S_{i-5}

! each new state is unanimously agreed and with an incremental counter to prevent playback attack

Application Specific State Channel



! Bob force the game to continue onchain by broadcasting the latest mutually signed state and his new move

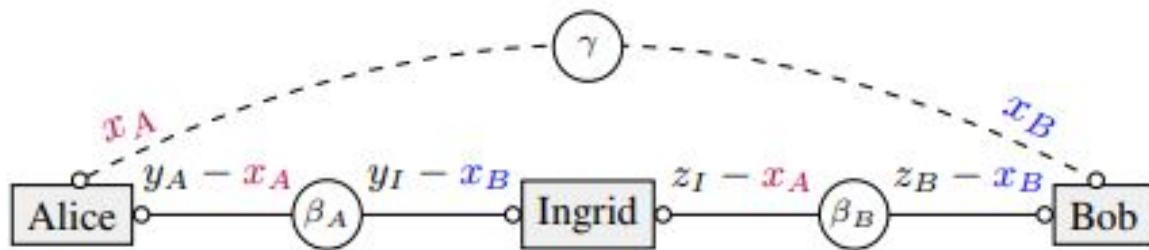
? Alice saw Bob have checkmated in S_{i+1} , thus become unresponsive

! each new state is unanimously agreed and with an incremental counter to prevent playback attack

Virtual Channel [Dziembowski et.al]

Advantage over routing via HTLC in Lightning:

Only involves intermediaries (hops) during opening and closing of "virtual channel", no cooperation needed during state update in-between



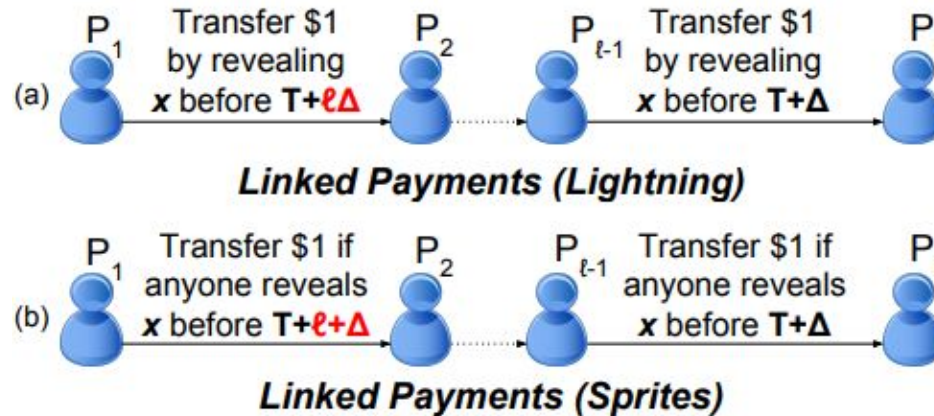
paper at <https://eprint.iacr.org/2017/635.pdf>

Sprite Channel [Miller et.al]

Core idea: use smart contract to

- Reduce worst-case collateral lock-up time
- Allow partial withdraw and deposit

$$\Delta \cdot l$$
$$\underbrace{t_1 + t_2 + t_3}_{A \rightarrow B}$$



Generalized State Channel

What if each channel is not app-specific?

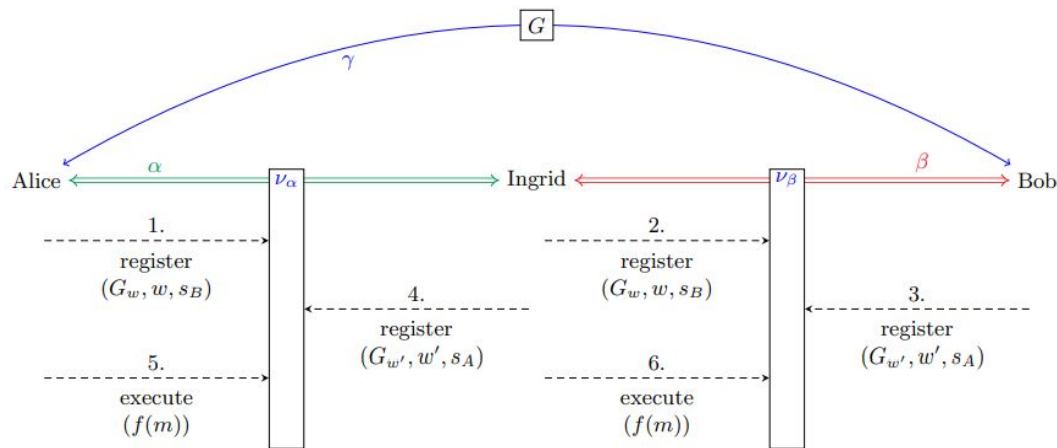
Core idea:

- Instantiate new contract offchain!! (via either opcode [create2](#) or contract registry mechanism s.t. contract can be deterministically addressed/referred to)
- Commit to contract code, start executing function via signed tx and signed post-computation states

Generalized State Channel

Core idea (cont):

- In case of dispute/misbehaving
 - Forcefully deploy the contract on-chain with latest signed states OR
 - Collapse from virtual channel to play out in a channel "one level up"



Learn More About GSC

- Perun Network: <https://www.perun.network/>
- StateChannel Team (prev: Counterfactual): <https://statechannels.org/>
- Connex Network: <https://connex.network/>
- Kristina's (from TU Darmstadt) presentation: <https://www.youtube.com/watch?v=kBptBui1wYs>
- Non-technical introduction by Perun: <https://perun.network/wp-content/uploads/Perun2.0.pdf>
- Formalized Generalized State Channel (with formal UC proof): <https://eprint.iacr.org/2018/320.pdf>
- Informal early proposal: <https://www.bgp4.com/wp-content/uploads/2019/05/statechannels.pdf>

Channel-based Network Summary



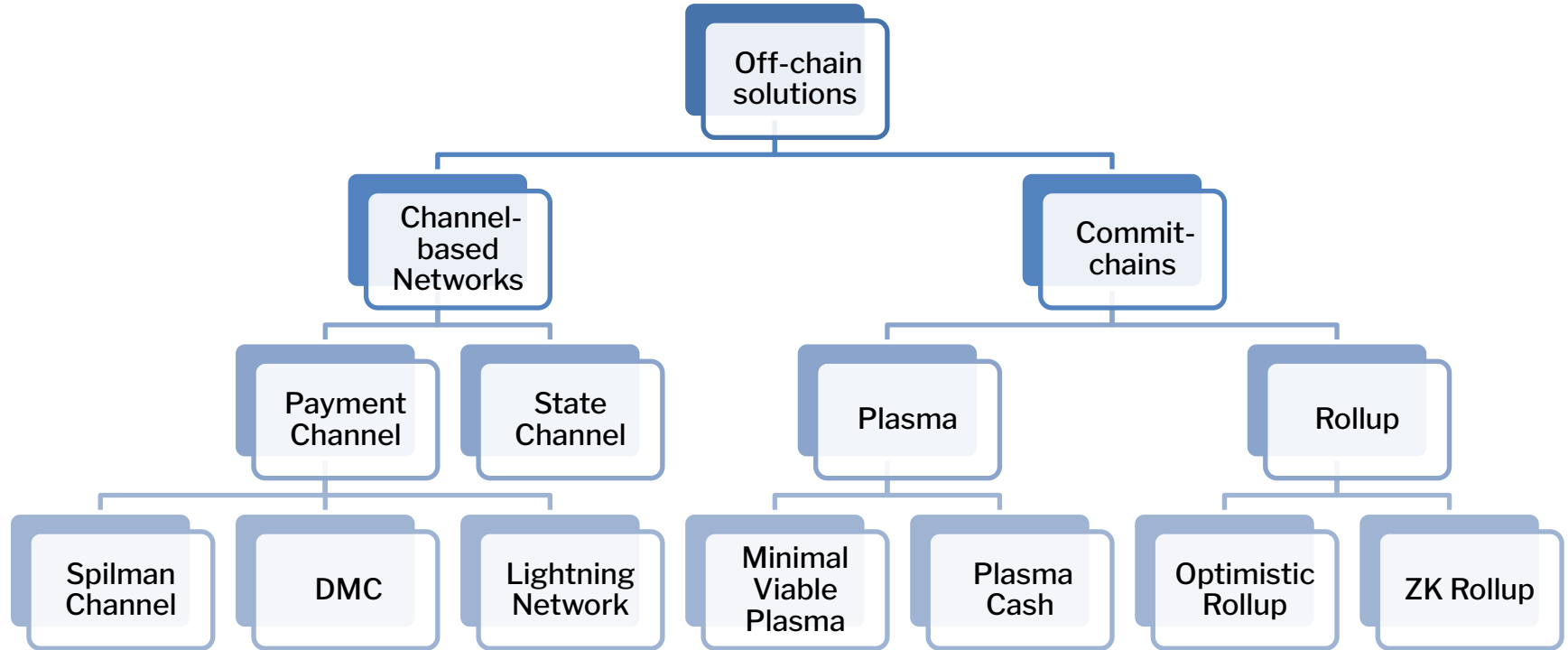
✓ Good:

- Instant finality
- Low cost (in most cases)
- Private – intermediate updates only known within the channel
- Best suited for a fixed group of participants with high transaction volume

✗ Bad:

- High collateral lockups for routing hub → capital inefficiency, potential point of failure
- **"always online" assumption** to ensure channel liveness and ensure fraud proof submission before dispute period elapse
- User are responsible for "data availability" relevant to themselves
- multi-hop/virtual channels are slightly complex during dispute resolution

Layer 2 Solutions



Channel Hub, Sidechain and Commit-chain

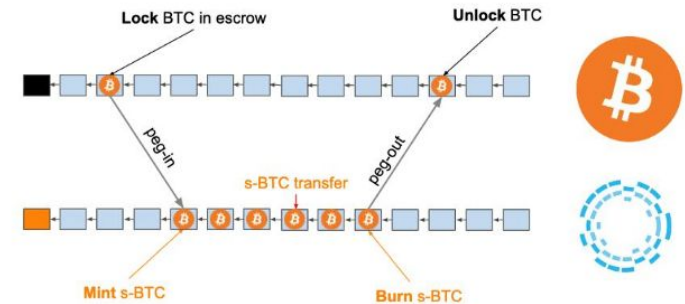
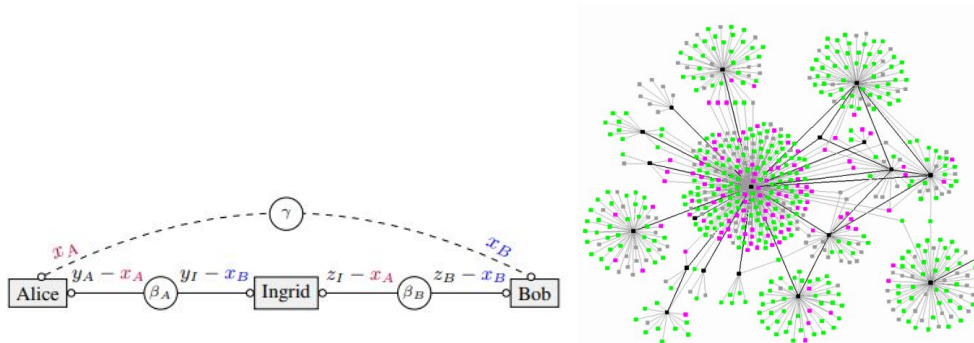
Complexity of routing will drive channel network into a "Hub-spoke" topology
Instant finality is great, but high collateral lockup required to be a hub compromise its "permissionless property"

Another class: sidechain uses trusted federation to facilitate 2-way peg into/out of another blockchain

Can we have a non-custodian sidechain ?

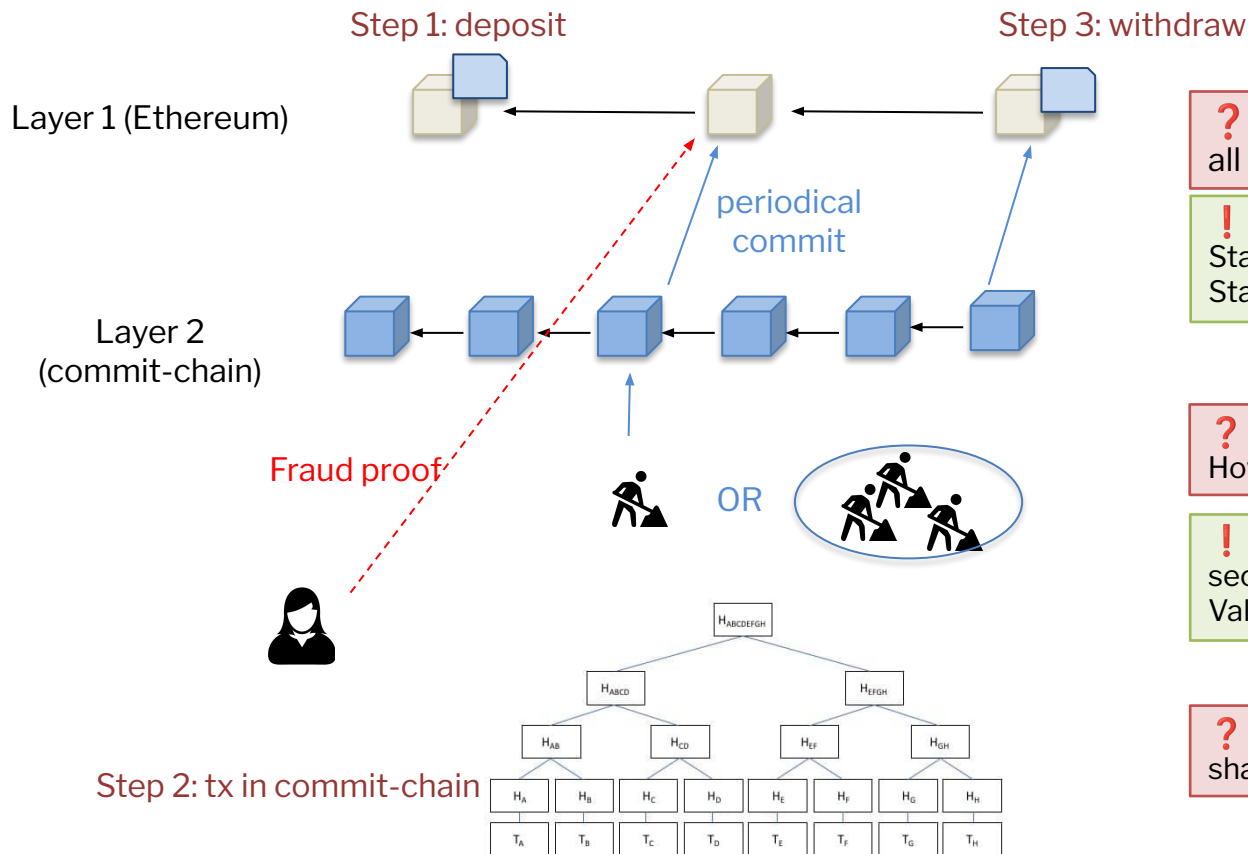
Can we relax finality to "eventual finality" to make layer 2 more permissionless?

Commit-chain



**federated pegs use multisig wallets for peg-in & peg-out*

Generic Commit-chain Workflow



Commit-chain
Manager Contract /
EthBridge Contract

? What to commit? State root only or all the transactions included?

! (roughly)
State root only (data offchain) → Plasma
State root + all tx (data onchain) → Rollup

? Who can be a validator in commit-chain?
How many of them is required?

! (mostly) anyone who put up enough security deposit in the EthBridge contract.
Validator set usually range from 1 to 100+

? This looks awfully similar to sharding scheme!! what's the difference??

Commit-chain v.s. Sharding

The architecture & approaches to ensure state validity and data availability are similar
Sharding requires "**tight coupling**" -- shared, controlled, maximized security among shard chains and beacon chain (via the same validator pool and randomly sampled & shuffled committees)

Commit-chain is much decoupled from the validity of the layer 1 chain even though two-way peg allows transfer asset between them; possibly only has a few or even a single validator

2 shard chains share the same spec whereas 2 commit-chain can be drastically different

Sharding	Try really hard to ensure total validity/availability of every part of the system
Plasma	Accept local faults but try to limit their consequences

More at: https://vitalik.eth.limo/general/2019/06/12/plasma_vs_sharding.html

Blockchain Scalability

Scaling Computation – Plasma

! once seemingly promising, now (sort of) abandoned with more new problems introduced than resolved.
Discuss for historical purpose & partial motivation to Rollup

History of Plasma



vitalik.eth
@VitalikButerin

And it begins... plasma.io @jcp

7:10 PM · Aug 9, 2017 · [Twitter Web App](#)

934 Retweets 2.1K Likes

Plasma: Scalable Autonomous S

Joseph Poon
joseph@lightning.network



Minimal Viable Plasma

Plasma new-extension



vbuterin

Special thanks to Joseph Poon and David Knott for discussions that

The following aims to provide a specification for a "minimal viable plasma implementation". It aims to provide the basic security properties of Plasma in a very simplified way, though it leans heavily on users being willing to immediately exit as soon as they detect any kind of malfeasance.

read more: <https://medium.com/dragonfly-research/the-life-and-death-of-plasma-b72c6a59c5ad>

Why Smart Contracts are NOT feasible on Plasma

Plasma



johba

While some MVP implementations are approaching production, more teams are exploring the realm of general computation on Plasma chains.

4 Jul '18

Vitalik Buterin

vitalik@

Basic Mass Exits for Plasma MVP

Plasma



kfichter

Introduction

The design of Plasma MVP requires that many UTXOs be withdrawn within a relatively short period of time (in the worst-case scenario). The maximum number of UTXOs that can be safely withdrawn inherently bounds the transactional capacity of the MVP chain. Currently, most MVP designs only

1 Sep '18



kfichter

Plasma XT embraces Plasma Cash's original vision of simple, reliable, low-cost transactions for everyone in the world.*

Plasma Cash: Plasma with much less per-user data checking

Plasma new-extension



vbuterin

[2018.03.10: updated with exit procedure]

4 Mar '18

discussion and coming up with much of this, as well as expressed similar ideas.

nomination payments in Plasma Cash

1 Jun '18

Cash, partially inspired by an on-going [Twitter](#) over 2 scaling solutions and how they relate to each other, as Buterin, @jcp, @karl, and others.

5 Jun '18

uanji, David Knott, Eva Beylin, Vitalik Buterin, and Kasima ns to putting this together.

ith much less per-user data checking

1 May '18



Minimal Viable Plasma

Plasma Contract on Ethereum as the bridge

Plasma Chain is maintained by Plasma operators/validators

Plasma MVP uses **UTXO** model for simplicity

Workflow:

- **Deposit** in Plasma Contract on Ethereum → add balance in Plasma chain
- Many transactions processed and included in a Plasma block by Plasma validator(s)
- Plasma block header gets committed to the Plasma Contract → transactions inside this block considered confirmed
- **Exit/Withdraw** by providing proof of UTXOs to Plasma Contract, after challenge period, if no successful dispute (proof of spent on UTXO claimed), then Plasma Contract release funds to the exited user

Users **frequently** check the Plasma chain, and exit when invalid blocks are detected

Plasma MVP Q&A

- What are being stored in the Plasma Contract?

The identities and security deposits of Plasma operators, all the Plasma block headers, logic for handling user deposit & withdraw & dispute

- How could user enforce timely balance top-up in Plasma chain when (s)he deposit?

When users deposit to the Plasma contract, the contract will immediately add one block header (containing the root node of a transaction tree with a single tx, mint UTXO to the user), since the Plasma contract represents the canonical history, Plasma operator must include the single-tx block thus effectively topping up to the users' balance

Plasma MVP Q&A

- The lifecycle of a payment in Plasma chain?

Alice creates a tx and signed it, send over to Plasma operator who verifies it and puts it into the next block together with other tx.

Plasma operator commit the new block header to Plasma contract, now the Alice's tx can be consider as *confirmed*.

BUT, she still need to send a "**confirm message**" which testify her confirmed tx to Bob for Bob to safely accept the payment and continue to transfer to others

(we will talk about why this wacky "confirmed message" is necessary soon, small hint, it has to do with annoying data unavailability)

Plasma MVP Q&A

- Why is Plasma secure? Can the operator steal money?

If the Plasma operator committed an invalid block by minting money for himself and withdrawing those money, then users would notice the bad behavior and all try to exit. Importantly, the Plasma contract handles withdraw in chronological order of UTXO, thus by the operator's withdraw claim's turn, **the contract is already drained** with no balance

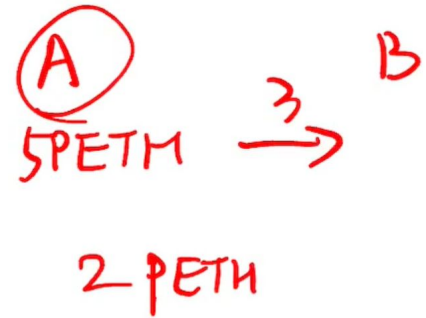
- Can a user (Bob) withdraw more than he owns in Plasma chain?

When Bob attempts to exit, he'll submit an "acclaimed UTXO" on Plasma contract and a merkle proof that it was included in one of the tx tree in history AND place a security deposit for this withdraw. This exit transaction will automatically start a **challenge period of 1 week**, during which others will submit fraud proofs (a tx signed by Bob that spends the claimed UTXO and the respective Merkle proof). If successful challenge exists, then Bob's security deposit goes to the challenger; else, Bob gets his security deposit refunded and his UTXO withdrawn

Plasma MVP's Awkward "Confirmed Message"

Reason:

1. Coin in Plasma MVP are fungible + Time order priority during exit
 - The order in which these exits are processed depends on the time they were requested
2. Block withholding by operator to trick user exit
 - "Block withholding" occurs when the operator intentionally does not publish a block or delays its publication
 - user tries to exit based on incomplete or incorrect information about their transaction's status.



read more: <https://ethresear.ch/t/why-do-dont-we-need-two-phase-sends-plus-confirmation/1866/13>

Plasma Cash

✓ Good:

Make each coin **non-fungible** (with unique serial number) to localize "potential damage"

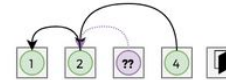
- to **avoid mass** exit
- to **minimize data checking by each user** (now only coins he/she owns)
- to avoid issues with "confirmed message"

✗ Bad:

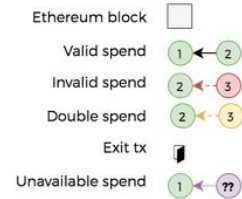
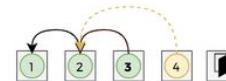
- No arbitrary denomination
- Ever-growing proof size for a coin
- Block withholding again 🧐

Block Withholding

Time 1: Block 3 is withheld and transaction 4 is valid from an honest user's perspective



Time 2: Block 3 becomes available and suddenly transaction 4 is a double spend



This is terrifying because once there is an unavailable block, users have no way to know if any subsequent spends are valid because an attacker can always make that block available at an inopportune time and invalidate their tokens. Terrible terrible terrible!

Plasma Links

- Best Plasma education site: <https://www.learnplasma.org/en/>
- Plasma MVP: <https://ethresear.ch/t/minimal-viable-plasma/426> ; Karl's presentation: https://www.youtube.com/watch?v=jTc_2tyT_IY
- Plasma
Cash: <https://ethresear.ch/t/plasma-cash-plasma-with-much-less-per-user-data-checking/1298> ; Karl's blog post on a simpler spec of Plasma
Cash: <https://karl.tech/plasma-cash-simple-spec/>